arXiv:2408.05868v3 [cs.CV] 30 May 2025

LaWa: Using Latent Space for In-Generation Image Watermarking

Ahmad Rezaei[1⋆], Mohammad Akbari[2], Saeed Ranjbar Alvar[2], Arezou Fatemi[2], and Yong Zhang[2]

[1] University of British Columbia
ahnr@mail.ubc.ca
[2] Huawei Technologies Canada Co. Ltd.
{mohammad.akbari,saeed.ranjbar.alvar1,yong.zhang3}@huawei.com

Abstract. With generative models producing high quality images that are indistinguishable from real ones, there is growing concern regarding the malicious usage of AI-generated images. Imperceptible image watermarking is one viable solution towards such concerns. Prior watermarking methods map the image to a latent space for adding the watermark. Moreover, Latent Diffusion Models (LDM) generate the image in the latent space of a pre-trained autoencoder. We argue that this latent space can be used to integrate watermarking into the generation process. To this end, we present LaWa, an in-generation image watermarking method designed for LDMs. By using coarse-to-fine watermark embedding modules, LaWa modifies the latent space of pre-trained autoencoders and achieves high robustness against a wide range of image transformations while preserving perceptual quality of the image. We show that LaWa can also be used as a general image watermarking method. Through extensive experiments, we demonstrate that LaWa outperforms previous works in perceptual quality, robustness against attacks, and computational complexity, while having very low false positive rate. Code is available here[3].

Keywords: Image Watermarking · Responsible AI · Image Generation

1 Introduction

With rapid advancements in generative models, AI-generated content (AIGC) in different modalities including images [41, 44, 47, 48], video [27, 53], text [8, 13, 57], and 3D [35, 43] can be generated with high quality. Text-to-image diffusion models such as Stable Diffusion [47] and DALL·E 2 [44] are open to public and have shown stunning performance in generation of photo-realistic images that are indistinguishable from real ones. Such tools can be misused in different ways such as faking AI-generated images as human-created artworks, generating fake news, impersonation, and copyright infringement [9, 59, 77]. Such threats raise concerns about our confidence and trust in the authenticity of photo-realistic

⋆ Work done during an internship at Huawei Technologies Canada Co. Ltd.
[3] https://github.com/vbdi/LaWa

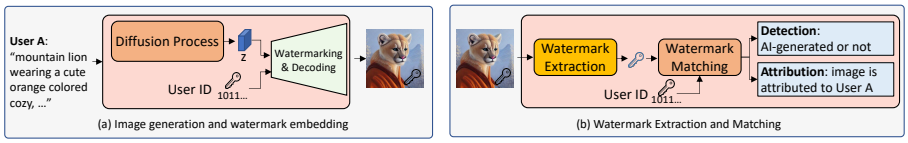


Fig. 1: An overview of in-generation image watermarking for LDMs. (a) generation and watermark (i.e., User ID) embedding. (b) Watermark extraction and matching for detection and attribution.

images. Thus, responsible implementation of generative AI services needs to be considered by service providers. Specifically, two problems including detection and attribution should be addressed. In detection, model developers detect if an image is generated by their model, i.e., if the image is AI-generated or not. In attribution, the developers attribute an image to the user who generated the image using their service.

Imperceptible image watermarking is a potential solution for the aforementioned problems. It involves embedding a message in the image without damaging its utility. A watermark extraction module is also required to accurately extract the embedded message from the watermarked image even if the it is modified. In the case of AI-generated images, the extracted message is used to detect such images and attribute them to the responsible user who created them (Fig. 1).

For AIGC images, existing watermarking methods can be applied after the images are generated (called post-generation methods in this paper) [20, 69]. Such post-generation watermarking introduces an overhead on the generation service due to the extra watermark post-processing. The watermarking process generally includes embedding the watermark into a latent feature representation of an image [20, 37, 60, 67]. The latent feature is either learned end-to-end in encoder-decoder-based watermarking methods [37,67] or it is extracted from the latent space of a pre-trained neural network [20,60]. In the case of Latent Diffusion Models (LDMs), generated images are in the latent space of a pre-trained autoencoder model. Thus, the latent features corresponding to the generation and watermarking procedures are de-coupled, which is a sub-optimal solution for combined image generation and watermarking problem.

In this paper, we propose LaWa, an in-generation watermarking method that effectively changes the latent feature of pre-trained LDMs to integrate watermarking into the generation process. Thus, generated images already conceal a watermark. Only a few prior works have addressed this problem [19, 62]. Stable Signature [19] is a model watermarking method for the decoder of LDMs. Since it requires fine-tuning a new decoder model for each watermark message, its application for in-generation watermarking has high computational cost and limited scalibility for an image generation service. Tree-Ring [62] watermarking modifies the initial noise vector used for image generation. It is only applicable to deterministic sampling [42] and requires image-to-noise inversion process for watermark extraction. Such process for text-guided diffusion does not always yield to the real noise vector [26, 39].

To address these limitations, LaWa incorporates novel coarse-to-fine multi-scale embedding modules into the frozen intermediate layers of the LDM decoder to ensure robustness against severe geometrical attacks. LaWa achieves high-payload in-generation watermarking with only one decoder model. Thus, it can handle many users for an image generation service with low computational cost. The focus of LaWa is on black-box watermarking, where users do not have access to the image generation model. In comparison with post-generation watermarking, we show that using the same latent features for image generation and watermarking improves the quality vs. robustness trade-off. Moreover, we show that LaWa can convert a pre-trained autoencoder into a robust general image watermarking method. In summary, our contributions are as follows:

- A multi-scale latent modification mechanism for pre-trained generative autoencoders that is compatible with any autoencoder and is robust to a broad range of image modifications
- A simple yet effective spatial watermark coding that improves the trade-off between robustness and perceptual quality
- An in-generation image watermarking approach that can be used for any pre-trained LDM and any image generation task without further fine-tuning of the LDM
- Extending the proposed in-generation watermarking method to work as a general post-generation image watermarking technique
- Achieving state-of-the-art post- and in-generation results in perceptual quality, robustness, and computational complexity.

2 Related Work

Detection of AI-Generated Images. Considering the risks of AIGC, many works focus on the passive detection of generated/manipulated images. These methods are well-studied for deep-fakes using inconsistencies in generated images [11,23,34] as well as generator traces in the spatial [38,64] or frequency [21, 74] domains. However, they have poor performance because they fall behind the rapid evolution of generative models. Similar approaches are proposed for diffusion models [14, 50], but they are also shown to suffer from low detection accuracy and high false rate [19].

Image Watermarking. There are three main categories of image watermarking methods. Transform-based methods embed the watermark in a spatial [22, 55] or frequency [28, 29, 40] domain, which can achieve great imperceptibility, but have low bit extraction robustness to even minor image modifications. Encoder-decoder-based methods use the encoder to concurrently create a latent feature and add the watermark before recreating the marked image [10,33,37,45,67,69,76]. Despite their good robustness, encoder-decoder networks may not generalize well to images out of the training data distribution and have strong trade-off between payload and utility. The third group uses the latent space of a fixed pre-trained network to add the watermark in several iterations [20,32,46,60,70], which substantially increases the watermarking time.

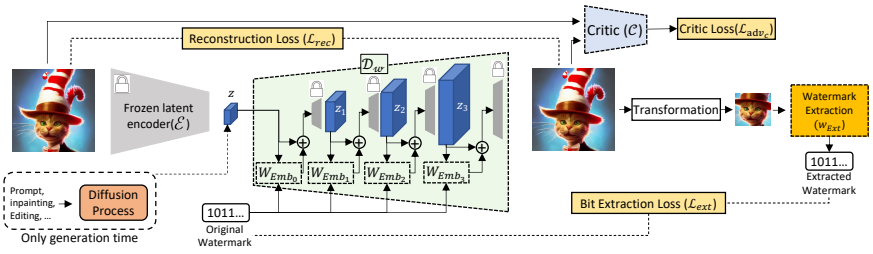


Fig. 2: Overall framework including the procedure of injecting the watermark embedding modules (W_{Emb}) to LDM's decoder ($\mathcal{D}$) as well as the end-to-end training process. During training, LDM's frozen encoder is used to generate latent z while at inference time, z is generated by the diffusion process.

RoSteALS proposes using latent space of autoencoders for steganography [10], but it does not consider image watermarking attack constraints. The proposed solution has no robustness to geometrical image modifications. Such limitation is critically important in the AIGC application scenario as users can easily evade the watermark.

In-Generation Image Watermarking. Previous works on integrating watermarking and image generation are mostly for model watermarking [58]. Some methods watermark the entire training dataset with one message [63, 65, 75] so that the output of the network carries the same message as well. Such methods are not extensible to new messages and require new training for every new message. Specifically for GANs, [66] proposes a solution to merge watermarking into the training procedure of the network. At the inference time, the generated images conceal a fingerprint specific to the user creating the image. This solution requires training the GAN model from scratch and is not applicable to pre-trained models.

In-generation watermarking for diffusion models is a recent research topic in the literature [19, 62]. Tree-Ring watermarking [62] modifies the initial noise vector used in image generation. This method cannot conceal a specific bit string into the image and is only applicable to deterministic sampling [42] in the diffusion process. For watermark extraction, an inversion process should accurately map the watermarked image to the initial noise. [26, 39] show that such inversion for text-guided diffusion does not always yield to the real noise vector, which can damage watermark extraction performance. Stable Signature [19] is a model watermarking method for the image decoder of a pre-trained LDM. If used for in-generation watermarking, a copy of the decoder model should be fine-tuned and stored for each watermark message, substantially increasing the computational cost and limiting the scalibility of this solution.

3 Methodology

The overall framework of how LaWa embeds and extracts messages is shown in Fig. 1 (in-generation watermarking) and Fig. 4 (image watermarking). In the

embedding phase, a latent is first generated by the diffusion process. Given the watermark message (i.e., user ID), LaWa enables the pre-trained LDM's latent decoder to simultaneously watermark and decode the latent, which results in generating the watermarked image. The watermark message can later be extracted using the extraction module to be used for detection and attribution by matching the message with one of existing watermarks.

Figure 2 illustrates the high-level procedure of modifying the LDM's decoder by injecting watermark embedding modules at intermediate layers of the decoder. The end-to-end training process of the added embedding and extraction networks is also shown, which will be described in details in the following subsections.

3.1 Modified Latent Decoder

In general, diffusion models are trained either in the image space [41, 48] or in a compact latent space [24, 44, 47] for more computational efficiency. For LDMs, the output of the diffusion process is in this latent space. Besides the diffusion process, LDMs also include an image autoencoder. The image encoder, $\mathcal{E}$, is used to downsample the input image $x \in \mathbb{R}^{H\times W\times 3}$ by a factor of f to a latent feature, z, where $z = \mathcal{E}(x) \in \mathbb{R}^{H/f\times W/f\times C}$ [47]. The decoder model, $\mathcal{D}$, then upsamples z to create the reconstructed image $\hat{x} = \mathcal{D}(\mathcal{E}(x))$. At the generation time, the diffusion process generates the latent feature z, which is mapped to the pixel space using the decoder: $\hat{x} = \mathcal{D}(z)$. The upsampling in $\mathcal{D}$ is a multi-step process in which latent size is scaled up by a factor of 2 after each step.

To embed the watermark information, we feed the watermark message into the latent feature before each upsampling step of $\mathcal{D}$. This results in a multi-scale coarse-to-fine process of embedding the watermark into the latent space features. More specifically, assuming a k-bit watermark message $m \in \{0,1\}^k$, our watermark embedding module at ith upsampling step, W_{Emb_i}, accepts m and the latent z_i as input and generates a residual latent, δz_i, with the same size as $z_i : \delta z_i = W_{Emb_i}(z_i, m)$, where $i \in \left\{0, \ldots, \frac{f}{2} - 1\right\}$ shows the upsampling step of $\mathcal{D}$. The watermarked latent, z_{w_i} is then constructed as:

$$z_{w_i} = z_i + \delta z_i. \tag{1}$$

As illustrated in Fig. 3, W_{Emb_i} comprises of a linear layer that maps m to a noise block $b_i \in \mathbb{R}^{B\times B\times C_i}$. C_i is the latent channel size at the ith step and B is the height and width size of b_i that is kept constant for all upsampling steps. After b_i is obtained, we perform spatial watermark coding where b_i is repeated along the height and width dimensions to match the size of z_i As a result of the applied spatial watermark coding, the watermark noise carries repeated watermark message with the idea to create sufficient redundancy in the noise to improve robustness to geometrical attacks while minimally damaging the perceptual quality. The watermark noise is then passed through a convolution layer to generate δz_i. To ensure W_{Emb_i} does not initially change the latent feature, we follow [71] and initialize the weights and biases of the convolution layer to 0.

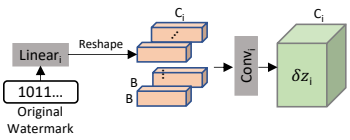


Fig. 3: Structure of the watermark embedding module W_{Emb_i}

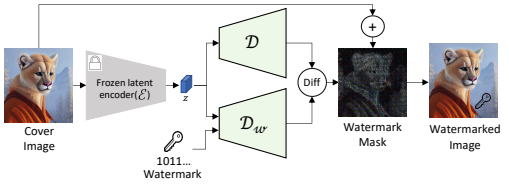


Fig. 4: Diagram of general image watermarking process with LaWa

Unlike the original decoder $\mathcal{D}$ that reconstructs the image without watermark $\hat{x} = \mathcal{D}(z)$, the modified latent decoder $\mathcal{D}_w$ generates the watermarked image $\hat{x}_w = \mathcal{D}_w(z)$.

3.2 Training

In the training process, the LDM's pre-trained image encoder as well as the original layers of the decoder are kept frozen. This ensures that training the watermark embedding modules do not degrade the image generation quality.

To ensure the perceptual similarity of the watermarked image $\hat{x}_w$ and the original generated image $\hat{x}$, we use a combination of pixel-wise distortion and perceptual loss functions. For the distortion loss, MSE (l_2 distance) loss defined as $\mathcal{L}_I = \|\hat{x}_w - \hat{x}\|^2$ is used. For the perceptual loss function, we employ LPIPS loss [72] to minimize the perceptual distortion between $\hat{x}_w$ and $\hat{x}$.

To further improve the quality of the watermarked image, an adversarial critic network $\mathcal{C}$ using Wasserstein GAN [5] is also implemented. Thus, the corresponding adversarial training includes two loss functions as follows:

$$\mathcal{L}_{adv\mathcal{D}_w} = -\mathbb{E}_{\hat{x}_w \sim P_{\hat{x}_w}}[\mathcal{C}(\hat{x}_w)], \tag{2}$$

$$\mathcal{L}_{adv\mathcal{C}} = \mathbb{E}_{\hat{x}_w \sim P_{\hat{x}_w}}[\mathcal{C}(\hat{x}_w)] - \mathbb{E}_{\hat{x} \sim P_{\hat{x}}}[\mathcal{C}(\hat{x})], \tag{3}$$

where $\mathcal{L}_{adv\mathcal{C}}$ and $\mathcal{L}_{adv\mathcal{D}_w}$ are the adversarial losses for the critic $\mathcal{C}$ and the modified decoder $\mathcal{D}_w$, respectively. $P_{\hat{x}_w}$ and $P_{\hat{x}}$ are the distribution of the watermarked and original images. The overall reconstruction loss is then defined as:

$$\mathcal{L}_{rec} = \lambda_I \mathcal{L}_I + \lambda_{LPIPS} \mathcal{L}_{LPIPS}(\hat{x}_w, \hat{x}) + \lambda_{adv} \mathcal{L}_{adv\mathcal{D}_w}, \tag{4}$$

where λ_I, λ_{LPIPS}, and λ_{adv} are the relative loss weights.

Our watermark extractor, denoted by W_{Ext}, should be robust to different image modifications and attacks. As a result, after the watermarked image is generated, a transformation T is randomly sampled from a set of differentiable transformations, $\mathcal{T}$, including common image processing attacks (see supplementary material for more details). For the differentiability issue of JPEG compression, we used the forward simulation attack introduced in [68]. The selected transformation is then applied to the image and the transformed image is passed through the watermark extraction network, which is based on ResNet50 [25] architecture with the last linear layer changed to output a k-bit message. The

output extracted message, $\hat{m} = W_{Ext}(T(\hat{x}))$, should match the original embedded message m. We define the extraction loss as the binary cross entropy (BCE) loss between m and $\hat{m}$, $\mathcal{L}_{\text{ext}} = \mathcal{L}_{BCE}(m, \hat{m})$. The final watermark embedding and extraction networks are then optimized to minimize the following total loss:

$$\mathcal{L} = \mathcal{L}_{rec} + \lambda \mathcal{L}_{ext}, \tag{5}$$

where λ is the loss weight to control the trade-off between extraction accuracy and image reconstruction quality.

3.3 Image Watermarking

Figure 4 shows the general image watermarking process. Corresponding latent feature z of the cover image x is calculated using the frozen encoder, $z = \mathcal{E}(x)$. The feature z is separately processed by $\mathcal{D}$ and $\mathcal{D}_w$ to create a watermark mask that is computed as $\mathcal{D}(z) - \mathcal{D}_w(z)$. Using this watermark mask, the cover image is then watermarked as $\hat{x}_w = x + \mathcal{D}_w(z) - \mathcal{D}(z)$.

3.4 Watermark Matching

Given an image at the watermark extraction time, our watermark extractor W_{Ext} tries to decode the watermark message $\hat{m}$. To address the detection and attribution problems, $\hat{m}$ needs to be matched with one of existing original messages (i.e., m) stored in the database. Following [19,65] and to increase the matching robustness against image modifications, we use a soft matching method defined as:

$$M(\hat{m}, m), \geq n \qquad n \in \{0, \ldots, k\}, \tag{6}$$

where $M(\hat{m}, m)$ counts the number of matching bits in $\hat{m}$ and m, and n is the threshold for flagging a match.

For a successful watermark matching, the rate of false detection and attribution should be at an acceptable level. Potential errors include detecting a vanilla image as watermarked (detection FPR), missing a watermarked image (detection false negative rate), and attributing an image to the wrong user (attribution FPR). Having low error rate is an important practical aspect of this problem.

Assuming that extracted bits from vanilla images are i.i.d Bernoulli random variables with parameter 0.5, the theoretical upper bound of FPR is as follows:

$$FPR_{det}(n) = Pr\left(M(\tilde{m}, m) > n \mid H_0\right) = \sum_{i=n+1}^{k} \binom{k}{i} 0.5^k, \tag{7}$$

$$FPR_{att}(N, n) = 1 - \left(1 - FPR_{det}(n)\right)^N, \tag{8}$$

where FPR_{det} is the detection FPR, FPR_{att} is the attribution FPR, N is the total number of users, $\tilde{m}$ is any random message, and H_0 is the null hypothesis: "any random message, $\tilde{m}$, match with m in more than n bits". Clearly, the more

the number of users, the higher the attribution FPR. By setting the FPR_{att} and total number of users to a desired level, we can obtain the minimum required n for watermark matching by using Eqs. (7) and (8) reversely. More information is provided in the supplementary material.

4 Experiments

In this section, the performance of the proposed method is quantitatively and qualitatively evaluated and compared with the previous works. We also study the generality of LaWa for different image generation tasks. A false positive analysis along with an extensive test for the trade-off between capacity, quality, and robustness is also presented. At the end, we will provide an ablation study to verify the effect of each loss term in our objective function.

4.1 Experimental Settings

To train LaWa's watermark embedding and extraction modules, we utilize 100K images from MIRFlickR dataset [30], where the images are randomly cropped to 256×256 resolution. The training is performed with an AdamW optimizer with the learning rate of $6e-5$, 40 epochs, and the batch size of 8. For all our experiments, we set the weights in Eqs. (4) and (5) to $\lambda_I = 0.1$, $\lambda_{LPIPS} = 1.0$, $\lambda_{adv} = 1.0$, and $\lambda = 2.0$, which are obtained experimentally. For the main experiments, we use the KL-f8 auto-encoder of Stable Diffusion [47] and 48-bit messages to train the LaWa method unless mentioned otherwise.

To numerically evaluate the distortion of the watermarked and original generated images, we use Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index Measure (SSIM) [61]. Moreover, we evaluate the perceptual quality of images using LPIPS similarity score [72] and Single Image Frechet Inception Distance (SIFID) [51].

4.2 Comparison Results

We compare the performance of LaWa with previous in-generation and image watermarking methods using AI-generated and natural image datasets. For AI-generated dataset, we consider using LaWa as both in-generation and post-generation watermarking methods. For the baseline of in-generation methods, we used Stable Signature (SS) [19], which is a model watermarking method for in-generation image watermarking by creating different copies of the decoder. Alternatively, image watermarking methods can be used to watermark the generated images in a post-generation manner. We provide the comparison results with different post-generation techniques including encoder-decoder based (HiDDeN [76], RivaGAN [69], and RoSteALS [10]), optimization-based (SSL [20] and FNNS [32]), and frequency-based (DCT-DWT [16]) methods. For HiDDeN and RivaGAN, 30-bit and 32-bit pre-trained weights in [31] and [52] are respectively used. For RoSteALS, we trained their model for 48- and 32-bit message lengths

Table 1: Comparison results of LaWa with existing post- and in-generation image watermarking methods in terms of quality, embedding time, and robustness against various attacks. *: in-generation methods.

Method (bit#)	Image quality		Emb. time (ms)	Bit accuracy ↑										
	PSNR/SSIM ↑	LPIPS/SIFID ↓		None	C. Crop 0.1	R. Crop 0.1	Resize 0.7	Rot. 15	Blur	Contr. 2.0	Bright. 2.0	JPEG 70	Comb.	Ave.
AI-generated Images														
DCT-DWT (32) [16]	39.47/0.97	0.03/0.02	139	0.91	0.51	0.51	0.52	0.52	0.51	0.52	0.51	0.51	0.51	0.55
Hidden (30) [31]	32.59/0.95	0.03/0.05	17	0.91	0.91	0.91	0.82	0.79	0.76	0.75	0.74	0.53	0.59	0.77
SSL (32) [20]	33.23/0.89	0.15/0.28	870	**1.00**	0.74	0.72	0.99	0.99	**1.00**	0.96	0.95	0.99	0.85	0.92
RoSteALS (32) [10]	29.31/0.93	0.04/0.06	144	**1.00**	0.50	0.51	**1.00**	0.47	**1.00**	0.88	0.86	**1.00**	0.50	0.77
RivaGan (32) [69]	**40.53/0.98**	0.03/0.04	57	0.99	0.98	**0.98**	0.87	0.91	0.99	0.81	0.80	0.98	0.93	0.92
LaWa* (32)	34.25/0.89	**0.03/0.01**	1	**1.00**	**1.00**	**0.98**	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	0.97	**0.99**
LaWa-post-gen (32)	34.28/0.90	0.03/0.02	33	**1.00**	**1.00**	0.96	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	**0.98**	**0.99**
SSL (48) [20]	33.25/0.89	0.15/0.27	870	**1.00**	0.72	0.70	0.99	**0.99**	**1.00**	0.94	0.94	0.99	0.82	0.91
FNNS (48) [32]	**36.84/0.97**	0.04/0.05	645	**1.00**	**0.98**	**0.96**	0.95	0.75	0.54	0.86	0.85	0.94	0.91	0.87
RoSteALS (48) [10]	29.30/0.92	0.05/0.07	144	**1.00**	0.51	0.50	**1.00**	0.50	**1.00**	0.88	0.85	**1.00**	0.49	0.78
SS* (48) [19]	32.02/0.84	0.05/0.09	0	0.99	0.95	0.93	0.96	0.81	0.78	0.97	0.96	0.92	0.92	0.92
LaWa* (48)	33.52/0.86	**0.04/0.02**	1	**1.00**	0.95	0.91	0.99	0.96	0.99	**1.00**	**1.00**	**1.00**	0.94	**0.97**
LaWa-post-gen (48)	**33.45/0.87**	**0.04/0.02**	34	**1.00**	0.94	0.90	0.96	0.97	0.99	**1.00**	**1.00**	**1.00**	**0.95**	**0.97**
CLIC														
DCT-DWT (32) [16]	38.94/0.97	**0.02/0.02**	230	0.87	0.51	0.51	0.51	0.51	0.52	0.50	0.50	0.52	0.50	0.55
Hidden (30) [31]	33.33/0.96	0.04/0.04	28	0.89	0.89	0.88	0.86	0.76	0.64	0.76	0.74	0.54	0.58	0.75
SSL (32) [20]	35.03/0.92	0.16/0.19	1261	**1.00**	0.86	0.80	**1.00**	**1.00**	**1.00**	0.96	0.93	0.98	0.90	0.94
RoSteALS (32) [10]	28.22/0.91	0.08/0.04	240	**1.00**	0.50	0.50	**1.00**	0.51	**1.00**	0.90	0.87	1.00	0.50	0.78
RivaGan (32) [69]	**42.04/0.98**	0.04/0.06	84	**1.00**	0.98	0.96	0.99	0.95	0.99	0.89	0.87	0.99	0.96	0.96
LaWa (32)	36.19/0.94	0.05/0.03	32	**1.00**	**1.00**	**0.97**	0.98	**1.00**	**1.00**	**1.00**	**0.99**	**1.00**	**0.99**	**0.99**
SSL (48) [20]	35.03/0.92	0.16/0.19	1261	**1.00**	0.83	0.78	**1.00**	**1.00**	**1.00**	0.96	0.92	0.98	0.88	0.93
FNNS (48) [32]	35.54/**0.96**	0.07/0.20	1116	**1.00**	0.99	0.95	0.94	0.75	0.51	0.85	0.82	0.91	0.89	0.86
RoSteALS (48) [10]	27.83/0.91	0.08/**0.04**	240	**1.00**	0.50	0.50	**1.00**	0.48	**1.00**	0.82	0.80	**1.00**	0.50	0.76
LaWa (48)	**35.58**/0.92	**0.05**/0.05	34	**1.00**	**1.00**	**0.97**	0.98	**0.98**	0.98	**1.00**	**0.99**	0.98	**0.90**	**0.98**

with the same dataset and attacks as in LaWa. For SSL and FNNS, we used their publicly available codes and their pre-trained modules to obtain the results.

The upper portion of Tab. 1 summarizes LaWa's results with 32- and 48-bit watermark messages compared to the baselines in terms of the watermarked image quality as well as the bit extraction accuracy after different attacks. The results are obtained by watermarking 1K generated images (with 1K text prompts obtained from MS-COCO [36] and MagicPrompt-SD [49] datasets). Resolution of generated images is set to 512×512. Each generated image is watermarked with 10 different messages and the average bit accuracy is reported. Combined attack includes a series of 40% area center cropping, brightness 2.0, and JPEG 80. LaWa* and LaWa-post-gen refer to the cases where the proposed method is used as in-generation and post-generation watermarking, respectively.

As shown in Tab. 1, LaWa achieves the best perceptual quality for AI-generated images with 32- and 48-bit versions in LPIPS and SIFID scores. For PSNR and SSIM, LaWa has comparable performance with other methods, while RivaGAN and FNNS outperform the others for 32- and 48-bit watermarks, respectively. For robustness to attacks, LaWa has the best bit accuracy for all attacks for 32-bit watermarks. For 48-bit watermarks, except for 10% random crop attack where FNNS works better (90% bit accuracy of LaWa compared to 96% of FNNS), LaWa has better or comparable bit accuracy. Overall, LaWa outperforms other methods in more number of attacks, specifically for the combined attack. Moreover, LaWa outperforms SS (as an in-generation method) in both image quality and robustness to a variety of attacks. More detailed set of

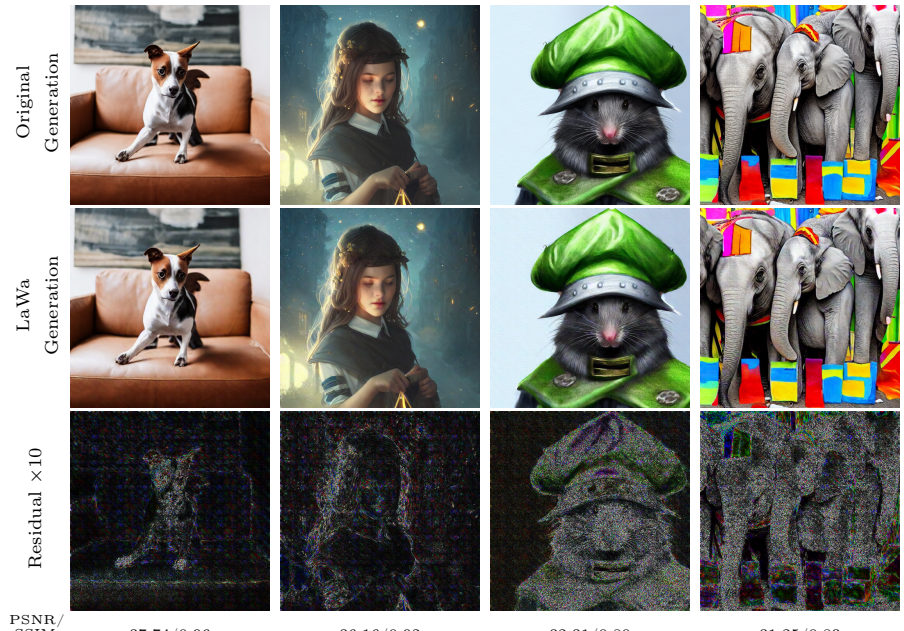


Fig. 5: Qualitative examples of generated images using LaWa, original generated images, and residual images (multiplied by 10). Extensive added noise by LaWa in high frequency textures reduces the PSNR/SSIM without damaging the perceptual quality (LPIPS/SIFID).

results with different attack parameters is given in the supplementary material. When LaWa is used as a post-generation watermarking method (i.e, decoupled from the generation), the extraction accuracy is nearly unchanged compared to in-generation watermarking.

The computational complexity of all methods compared to ours in terms of averaged embedding time is also given in Tab. 1. Overall, it is seen that post-generation methods result in a significant overhead for watermarking the generated images, especially the iteration-based ones such as SSL and FNNS. On the other hand, the in-generation methods such as SS and ours provide near zero overhead as no post-processing is needed to add the watermark.

We also compare the performance of LaWa in general image watermarking against baseline methods using a natural (i.e, not AI-generated) dataset. Following earlier works in [10, 20], CLIC dataset [56] is used, which has 428 test images with different resolutions. For all images, while maintaining the aspect ratio, larger dimension of each image is resized to 1024 to avoid memory limitation issue for some of the baselines. The results with CLIC dataset are given in the bottom part of Tab. 1. LaWa-post-gen achieves the highest bit accuracy for almost all the attacks and comparable or better visual quality compared to the baselines. Qualitative results for CLIC are in the supplementary material.

4.3 Qualitative Analysis

Fig. 5 shows visual examples of 4 images generated by the original and modified decoders (i.e., original generated vs. watermarked images). The residual images (multiplied by 10) and the numerical quality metrics including PSNR, SSIM, LPIPS, and SIFID are also given for each pair.

Comparing the watermarked and original images, although the calculated PSNR is not very high, it is visually not possible to differentiate them as no visual artifact is evident. This is expected as we trained LaWa focusing on the perceptual quality of the watermarked images while giving lower weight to the MSE loss, which directly affects the PSNR. From the examples in Fig. 5, it is clear that our method has learned to add more noise to the high frequency textures, especially for the generated images with more texture (e.g., the elephants). In practice, when using in-generation watermarking, there is no original image available for comparison as all generated images conceal the watermark. This relaxes the requirement for watermarked images to have a high pixel-wise similarity to the original image. Therefore, we argue that if generated images have no visual artifact, watermark addition is successful even if PSNR is not very high. More qualitative results compared to the previous works are presented in the supplementary material.

4.4 Capacity, Quality, and Robustness Trade-off

Effect of Number of Bits. We study the effect of the number of bits on generation quality and robustness in Tab. 2. LaWa can hide 64- and 96-bit watermark messages with minor drop in image quality, while achieving near 100% bit accuracy with no attack. With 128-bit messages, minor drop in the quality and bit accuracy against attack is observed compared to embedding shorter messages. Using higher number of bits can further improve the FPR of LaWa for its application in image generation services, which are designed to support a large number of users.

Table 2: Effect of the number of bits on generation quality and bit extraction accuracy.

Bit length	PSNR/ SSIM	LPIPS/ SIFID	None	C. Cro 0.1	Contr. 2.0	Rot. 15	JPEG 70	Comb.
32	34.25/0.89	0.03/0.01	1.00	1.00	1.00	1.00	1.00	0.97
48	33.52/0.86	0.04/0.02	1.00	0.95	1.00	0.96	1.00	0.94
64	33.11/0.83	0.04/0.02	1.00	0.93	0.99	0.92	0.96	0.95
96	32.82/0.83	0.04/0.02	0.99	0.87	0.98	0.77	0.92	0.95
128	31.87/0.82	0.08/0.09	0.95	0.93	0.95	0.84	0.92	0.87

Quality-Robustness Trade-off. We can trade the generation quality for higher bit accuracy by changing λ in Eq. (5). Table 3 presents the performance of LaWa when trained with different λs. The corresponding PSNR and bit accuracy (after applying combined attack) on 1K images is reported. As observed, by increasing λ, the extraction robustness is improved, but the quality of the watermarked images drops.

Table 3: Trade-off between image quality and bit accuracy. λ: trade-off weight in Eq. (5).

λ	1.0	1.7	2.0	4.0	7.0	10.0
PSNR ↑	34.76	33.84	33.52	33.06	30.56	28.47
Bit acc. on comb. ↑	0.65	0.75	0.94	0.96	0.97	0.99

Table 4: Ablation study on the effect of image quality loss weights.

λ_I	0.1	0.5	1.0
PSNR/SSIM ↑	33.52/0.86	33.60/0.88	**33.94/0.90**
LPIPS/SIFID ↓	**0.04/0.02**	0.05/0.04	0.06/0.05
Bit acc. on None/comb. ↑	**1.00/0.94**	1.00/0.88	0.99/0.84

4.5 LaWa's Performance for Different Tasks

In this sub-section, the generality of LaWa for different image generation tasks [47] including text-to-image, inpainting, super-resolution, and image editing is studied. The corresponding performance results are given in Tab. 5. The results are obtained for 1K images generated/watermarked with a separate random watermark message of 48 bits (see the supplementary material for more details). As summarized in the Tab. 5, the achieved bit accuracy averaged over all the tasks is higher than 97%, while the perceptual quality is preserved. This shows the applicability and generality of our proposed method to different generation tasks.

4.6 False Positive Analysis

The false positive analysis of LaWa's performance for both detection and attribution problems is given in this sub-section. As discussed in Sec. 3.4, in the detection problem, we only aim to detect if an image is watermarked while having a low FPR (detecting vanilla images as watermarked). On the other hand, in the attribution problem, we aim to find the user who created the image without making a false attribution to the wrong user (false attribution).

Detection. For detection, we use 1K watermarked images with a single 48-bit message to study the effect of the threshold n used for soft watermark matching on FPR_{det}. For each $n \in \{0, ..., 48\}$, we use Eq. (7) to calculate FPR_{det}, while TPR_{det} is experimentally obtained for the 1K images under various attacks. We experimentally validate the theoretical FPR_{det} in the supplementary material.

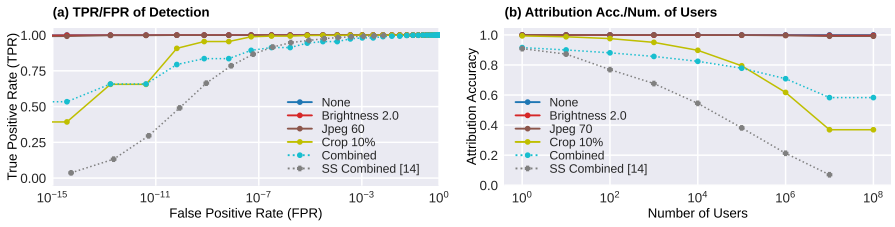


Fig. 6: Watermark matching performance in detection and attribution. (a) Detection TPR/FPR for different number of matching bits under different attacks. (b) Accuracy of attributing the watermarked image to the right user under different attacks and number of users assuming the detection FPR of 10^{-6}.

Table 5: Performance of LaWa in different image generation tasks in terms of quality and robustness.

Task	PSNR/ SSIM	LPIPS/ SIFID	C. Crop 0.1	Contr. 2.0	Rot. 15	JPEG 70
Text-to-Image	33.52/0.86	0.04/0.02	0.95	1.00	0.96	1.00
Inpainting	33.68/0.86	0.04/0.02	0.94	1.00	0.97	0.99
Super-Resolution	35.40/0.89	0.04/0.01	0.96	0.99	0.96	0.98
Image Editing	34.22/0.88	0.04/0.01	0.95	1.00	0.96	0.99

Table 6: Ablation study on the effect of each loss term on image quality and bit extraction robustness.

MSE	LPIPS	Critic	PSNR/SSIM	LPIPS/SIFID	Bit Acc. Comb.
✓			31.26/0.82	0.22/0.52	0.96
	✓		28.83/0.36	0.05/0.41	0.93
✓	✓		29.47/0.47	0.04/0.31	0.85
✓		✓	32.29/0.80	0.10/0.05	**0.99**
	✓	✓	33.05/0.85	0.05/0.02	0.73
✓	✓	✓	**33.52/0.86**	**0.04/0.02**	0.94

Figure 6a shows LaWa performance at different FPR_{det}. Under no attack, brightness 2.0, and JPEG 60, LaWa successfully detects 99% of the images while only flagging 1 out of 10^{15} vanilla images as watermarked. For more severe attacks such as crop 10% and combined attacks, there is a clear trade-off between detection accuracy and FPR_{det}. LaWa still detects more than 95% of the cropped and 83% of the combined-attacked images while only flagging 1 out of 10^9 vanilla images. We also compare LaWa with Stable Signature (SS) [19] under combined attack and show that LaWa has a much higher TPR.

Attribution. For the attribution problem, we can use Eqs. (7) and (8) reversely to find the number of possible users for any FPR_{att}. For example, for $FPR_{att} = 10^{-6}$ in Eq. (8), we can calculate the required FPR_{det} for any number of users N. We further use the computed FPR_{det} and Eq. (7) to find the necessary watermark matching threshold n. For example, for $FPR_{att} = 10^{-6}$ and $N = 10^5$, we should use $n = 45$ as the threshold of soft watermark matching for 48-bit watermark messages.

We study LaWa's attribution accuracy for each N with 50K different 48-bit messages. We generate 2 images per message resulting in a total of 100K images. For $FPR_{att} = 10^{-6}$ and each N, the required threshold n is computed. Using this n, we perform watermark matching between the extracted message from each image and all the other messages. The match with the highest score above n is considered true positive (TP) if it corresponds to the correct user. Accuracy is calculated as $TP/100k$. For Ns larger than 50k, we assume some users have not generated any image.

Figure 6b shows the attribution accuracy of our method for different attacks. Under no attack, brightness 2.0, and JPEG 70, LaWa attributes 99% of the generated images to the correct user among 10^6 total users. Accuracy decreases to less than 70% and 60% for more severe combined and 10% crop attacks, respectively. Compared to Stable Signature (SS) [19], LaWa achieves a better performance with 20% accuracy for the combined attack. Moreover, LaWa has zero false attribution when $N = 10^6$, which means no image was attributed to the wrong user. This is expected as the watermark matching threshold is really high at $n = 47$. Overall, as we increase the number of users, the probability of false accusations also increases.

4.7 Learning-based Attacks

We study the impact of learning-based watermark removal attacks, where the watermarked image is passed through an auto-encoder model [4, 6, 12, 18, 47] as an attempt to remove the watermark. Figure 7 shows the robustness of LaWa compared to Stable Signature (SS) [19] against several auto-encoder models including Bmshj2018 [6], Cheng2020 [12], KL-VAE [47], and VQ-VAE [18].

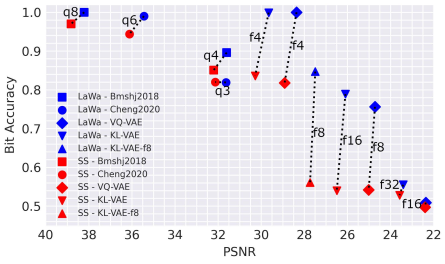


Fig. 7: Effect of the learning-based removal attacks. Results are reported for (SS) [19] and LaWa. q, f: quality/downsampling factors for each attack.

Each type of attack is applied using different quality/downsampling factors used in the auto-encoders. We observe that LaWa is quite robust against the attacks performed by Bmshj2018 with q8, Cheng2020 with q6, KL-VAE with f4, and VQ-VAE with f4. As the attack gets stronger (i.e., images are more compressed with larger quality or downsampling factors), both the bit accuracy and the PSNR reduce. Meaning if the attack is strong enough to remove the watermark, the quality of the watermarked image will inevitably experience a significant degradation (i.e., PSNR<24). It is also observed that even when the auto-encoder used in the attack is the same as the one we used to generate/watermark the images (i.e., KL-VAE f8), the attack will not erase the watermark and the bit accuracy remains as high as 85%. In contrast, the bit accuracy of SS [19] drops to 55%. It should be noted that unlike [19], we calculate the PSNR between the watermarked and the attacked images which is a more accurate representation of the attack's effect.

4.8 Ablation Study

Loss Terms. Table 6 shows the effect of each loss used in Eq. (4). We observe that the Critic network helps to improve the visual quality of the images. Using MSE and Critic losses achieves the best bit accuracy, but this setting leads to low perceptual quality and high visual artifacts in the image. By adding LPIPS loss, such visual artifacts are removed from the generated images.

Loss Weights. Table 4 shows the effect of changing loss weights in Eq. (4). We did this study by fixing both λ_{LPIPS} and λ_{adv} to 1.0 and changing λ_I. We observe that $\lambda_I > 0.1$ provides minor improvement in PSNR/SSIM, but damages LPIPS/SIFID and LaWa's robustness. More ablation studies over other components of LaWa along with qualitative results are given in the supplementary material.

5 Conclusion

In this paper, we proposed an in-generation image watermarking solution, which is applicable to LDMs and any image generation task. We showed that by injecting the watermark embedding modules into the intermediate layers of the

frozen decoder, these modules can learn the decoder-specific mapping of latent space to RGB images. Thus, the watermark message is concealed in the generated images in a way that is imperceptible and robust to image modifications. Our solution is scalable to support a large number of users only with one single decoder model. With the fast evolution of generative AI models and its corresponding risks, model developers are expected to actively help with the safe adaptation of these models. Our work is a step towards responsible AI and we hope it can help model developers to implement their generative AI service while they have countermeasures in place to limit any improper use.

References

1. Stable diffusion v1.4. `https://github.com/CompVis/stable-diffusion` (2021)
2. Stable diffusion v2.1. `https://github.com/Stability-AI/StableDiffusion` (2022)
3. Agustsson, E., Timofte, R.: Ntire 2017 challenge on single image super-resolution: Dataset and study. In: Proceedings of the IEEE conference on computer vision and pattern recognition workshops. pp. 126–135 (2017)
4. Akbari, M., Liang, J., Han, J., Tu, C.: Learned bi-resolution image coding using generalized octave convolutions. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 35, pp. 6592–6599 (2021)
5. Arjovsky, M., Chintala, S., Bottou, L.: Wasserstein gan (2017)
6. Ballé, J., Minnen, D., Singh, S., Hwang, S.J., Johnston, N.: Variational image compression with a scale hyperprior (2018)
7. Bégaint, J., Racapé, F., Feltman, S., Pushparaja, A.: Compressai: a pytorch library and evaluation platform for end-to-end compression research. arXiv preprint arXiv:2011.03029 (2020)
8. Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al.: Language models are few-shot learners. Advances in neural information processing systems **33**, 1877–1901 (2020)
9. Brundage, M., Avin, S., Clark, J., Toner, H., Eckersley, P., Garfinkel, B., Dafoe, A., Scharre, P., Zeitzoff, T., Filar, B., et al.: The malicious use of artificial intelligence: Forecasting, prevention, and mitigation. arXiv preprint arXiv:1802.07228 (2018)
10. Bui, T., Agarwal, S., Yu, N., Collomosse, J.: Rosteals: Robust steganography using autoencoder latent space. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 933–942 (2023)
11. Chai, L., Bau, D., Lim, S.N., Isola, P.: What makes fake images detectable? understanding properties that generalize. In: Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XXVI 16. pp. 103–120. Springer (2020)
12. Cheng, Z., Sun, H., Takeuchi, M., Katto, J.: Learned image compression with discretized gaussian mixture likelihoods and attention modules. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (2020)
13. Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H.W., Sutton, C., Gehrmann, S., et al.: Palm: Scaling language modeling with pathways. arXiv preprint arXiv:2204.02311 (2022)

14. Corvi, R., Cozzolino, D., Zingarini, G., Poggi, G., Nagano, K., Verdoliva, L.: On the detection of synthetic images generated by diffusion models. In: ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). pp. 1–5. IEEE (2023)
15. Couairon, G., Verbeek, J., Schwenk, H., Cord, M.: Diffedit: Diffusion-based semantic image editing with mask guidance. arXiv preprint arXiv:2210.11427 (2022)
16. Cox, I., Miller, M., Bloom, J., Fridrich, J., Kalker, T.: Digital watermarking and steganography. Morgan kaufmann (2007)
17. Deng, J., Dong, W., Socher, R., Li, L.J., Li, K., Fei-Fei, L.: Imagenet: A large-scale hierarchical image database. In: 2009 IEEE conference on computer vision and pattern recognition. pp. 248–255. Ieee (2009)
18. Esser, P., Rombach, R., Ommer, B.: Taming transformers for high-resolution image synthesis (2021)
19. Fernandez, P., Couairon, G., Jégou, H., Douze, M., Furon, T.: The stable signature: Rooting watermarks in latent diffusion models. ICCV (2023)
20. Fernandez, P., Sablayrolles, A., Furon, T., Jégou, H., Douze, M.: Watermarking images in self-supervised latent spaces. In: ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). pp. 3054–3058. IEEE (2022)
21. Frank, J., Eisenhofer, T., Schönherr, L., Fischer, A., Kolossa, D., Holz, T.: Leveraging frequency analysis for deep fake image recognition. In: International conference on machine learning. pp. 3247–3258. PMLR (2020)
22. Ghazanfari, K., Ghaemmaghami, S., Khosravi, S.R.: Lsb++: An improvement to lsb+ steganography. In: TENCON 2011-2011 IEEE Region 10 Conference. pp. 364–368. IEEE (2011)
23. Gragnaniello, D., Cozzolino, D., Marra, F., Poggi, G., Verdoliva, L.: Are gan generated images easy to detect? a critical analysis of the state-of-the-art. In: 2021 IEEE international conference on multimedia and expo (ICME). pp. 1–6. IEEE (2021)
24. Gu, S., Chen, D., Bao, J., Wen, F., Zhang, B., Chen, D., Yuan, L., Guo, B.: Vector quantized diffusion model for text-to-image synthesis. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 10696–10706 (2022)
25. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proceedings of the IEEE conference on computer vision and pattern recognition. pp. 770–778 (2016)
26. Hertz, A., Mokady, R., Tenenbaum, J., Aberman, K., Pritch, Y., Cohen-Or, D.: Prompt-to-prompt image editing with cross attention control. arXiv preprint arXiv:2208.01626 (2022)
27. Ho, J., Chan, W., Saharia, C., Whang, J., Gao, R., Gritsenko, A., Kingma, D.P., Poole, B., Norouzi, M., Fleet, D.J., et al.: Imagen video: High definition video generation with diffusion models. arXiv preprint arXiv:2210.02303 (2022)
28. Holub, V., Fridrich, J.: Designing steganographic distortion using directional filters. In: 2012 IEEE International workshop on information forensics and security (WIFS). pp. 234–239. IEEE (2012)
29. Holub, V., Fridrich, J., Denemark, T.: Universal distortion function for steganography in an arbitrary domain. EURASIP Journal on Information Security **2014**, 1–13 (2014)
30. Huiskes, M.J., Lew, M.S.: The mir flickr retrieval evaluation. In: MIR '08: Proceedings of the 2008 ACM International Conference on Multimedia Information Retrieval. ACM, New York, NY, USA (2008)

31. Khachatryan, A.: Hidden. https://github.com/ando-khachatryan/HiDDeN (2019)
32. Kishore, V., Chen, X., Wang, Y., Li, B., Weinberger, K.Q.: Fixed neural network steganography: Train the images, not the network. In: International Conference on Learning Representations (2021)
33. Lee, J.E., Seo, Y.H., Kim, D.W.: Convolutional neural network-based digital image watermarking adaptive to the resolution of image and watermark. Applied Sciences **10**(19), 6854 (2020)
34. Li, Y., Lyu, S.: Exposing deepfake videos by detecting face warping artifacts. arXiv preprint arXiv:1811.00656 (2018)
35. Lin, C.H., Gao, J., Tang, L., Takikawa, T., Zeng, X., Huang, X., Kreis, K., Fidler, S., Liu, M.Y., Lin, T.Y.: Magic3d: High-resolution text-to-3d content creation. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 300–309 (2023)
36. Lin, T.Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., Zitnick, C.L.: Microsoft coco: Common objects in context. In: Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part V 13. pp. 740–755. Springer (2014)
37. Luo, X., Zhan, R., Chang, H., Yang, F., Milanfar, P.: Distortion agnostic deep watermarking. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. pp. 13548–13557 (2020)
38. Marra, F., Gragnaniello, D., Verdoliva, L., Poggi, G.: Do gans leave artificial fingerprints? In: 2019 IEEE conference on multimedia information processing and retrieval (MIPR). pp. 506–511. IEEE (2019)
39. Mokady, R., Hertz, A., Aberman, K., Pritch, Y., Cohen-Or, D.: Null-text inversion for editing real images using guided diffusion models. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 6038–6047 (2023)
40. Navas, K., Ajay, M.C., Lekshmi, M., Archana, T.S., Sasikumar, M.: Dwt-dct-svd based watermarking. In: 2008 3rd International Conference on Communication Systems Software and Middleware and Workshops (COMSWARE'08). pp. 271–274. IEEE (2008)
41. Nichol, A., Dhariwal, P., Ramesh, A., Shyam, P., Mishkin, P., McGrew, B., Sutskever, I., Chen, M.: Glide: Towards photorealistic image generation and editing with text-guided diffusion models. arXiv preprint arXiv:2112.10741 (2021)
42. Nichol, A.Q., Dhariwal, P.: Improved denoising diffusion probabilistic models. In: International Conference on Machine Learning. pp. 8162–8171. PMLR (2021)
43. Poole, B., Jain, A., Barron, J.T., Mildenhall, B.: Dreamfusion: Text-to-3d using 2d diffusion. arXiv (2022)
44. Ramesh, A., Dhariwal, P., Nichol, A., Chu, C., Chen, M.: Hierarchical text-conditional image generation with clip latents. arXiv preprint arXiv:2204.06125 **1**(2), 3 (2022)
45. Ranjbar Alvar, S., Akbari, M., Chu, L., Zhang, Y., et al.: Amuse: Adaptive multi-segment encoding for dataset watermarking. arXiv preprint arXiv:2403.05628 (2024)
46. Ranjbar Alvar, S., Akbari, M., Yue, D., Zhang, Y.: Nft-based data marketplace with digital watermarking. In: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. pp. 4756–4767 (2023)
47. Rombach, R., Blattmann, A., Lorenz, D., Esser, P., Ommer, B.: High-resolution image synthesis with latent diffusion models. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. pp. 10684–10695 (2022)

48. Saharia, C., Chan, W., Saxena, S., Li, L., Whang, J., Denton, E.L., Ghasemipour, K., Gontijo Lopes, R., Karagol Ayan, B., Salimans, T., et al.: Photorealistic text-to-image diffusion models with deep language understanding. Advances in Neural Information Processing Systems **35**, 36479–36494 (2022)
49. Santana, G.: magicprompt. `https://huggingface.co/Gustavosta/MagicPrompt-Stable-Diffusion` (2022)
50. Sha, Z., Li, Z., Yu, N., Zhang, Y.: De-fake: Detection and attribution of fake images generated by text-to-image diffusion models. arXiv preprint arXiv:2210.06998 (2022)
51. Shaham, T.R., Dekel, T., Michaeli, T.: Singan: Learning a generative model from a single natural image. In: Proceedings of the IEEE/CVF international conference on computer vision. pp. 4570–4580 (2019)
52. ShieldMnt: invisible-watermark. `https://github.com/ShieldMnt/invisible-watermark/tree/main` (2021)
53. Singer, U., Polyak, A., Hayes, T., Yin, X., An, J., Zhang, S., Hu, Q., Yang, H., Ashual, O., Gafni, O., et al.: Make-a-video: Text-to-video generation without text-video data. arXiv preprint arXiv:2209.14792 (2022)
54. Suvorov, R., Logacheva, E., Mashikhin, A., Remizova, A., Ashukha, A., Silvestrov, A., Kong, N., Goka, H., Park, K., Lempitsky, V.: Resolution-robust large mask inpainting with fourier convolutions. In: Proceedings of the IEEE/CVF winter conference on applications of computer vision. pp. 2149–2159 (2022)
55. Taha, M.S., Rahem, M.S.M., Hashim, M.M., Khalid, H.N.: High payload image steganography scheme with minimum distortion based on distinction grade value method. Multimedia Tools and Applications **81**(18), 25913–25946 (2022)
56. Toderici, G., Shi, W., Timofte, R., Theis, L., Balle, J., Agustsson, E., Johnston, N., Mentzer, F.: Workshop and challenge on learned image compression (clic2020). In: CVPR (2020)
57. Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al.: Llama: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971 (2023)
58. Uchida, Y., Nagai, Y., Sakazawa, S., Satoh, S.: Embedding watermarks into deep neural networks. In: Proceedings of the 2017 ACM on international conference on multimedia retrieval. pp. 269–277 (2017)
59. Vincent, J.: An online propaganda campaign used ai-generated headshots to create fake journalists. Verge. com (2020)
60. Vukotić, V., Chappelier, V., Furon, T.: Are deep neural networks good for blind image watermarking? In: 2018 IEEE International Workshop on Information Forensics and Security (WIFS). pp. 1–7. IEEE (2018)
61. Wang, Z., Bovik, A.C., Sheikh, H.R., Simoncelli, E.P.: Image quality assessment: from error visibility to structural similarity. IEEE transactions on image processing **13**(4), 600–612 (2004)
62. Wen, Y., Kirchenbauer, J., Geiping, J., Goldstein, T.: Tree-ring watermarks: Fingerprints for diffusion images that are invisible and robust. arXiv preprint arXiv:2305.20030 (2023)
63. Wu, H., Liu, G., Yao, Y., Zhang, X.: Watermarking neural networks with watermarked images. IEEE Transactions on Circuits and Systems for Video Technology **31**(7), 2591–2601 (2020)
64. Yu, N., Davis, L.S., Fritz, M.: Attributing fake images to gans: Learning and analyzing gan fingerprints. In: Proceedings of the IEEE/CVF international conference on computer vision. pp. 7556–7566 (2019)

65. Yu, N., Skripniuk, V., Abdelnabi, S., Fritz, M.: Artificial fingerprinting for generative models: Rooting deepfake attribution in training data. In: Proceedings of the IEEE/CVF International conference on computer vision. pp. 14448–14457 (2021)
66. Yu, N., Skripniuk, V., Chen, D., Davis, L., Fritz, M.: Responsible disclosure of generative models using scalable fingerprinting. In: International Conference on Learning Representations (ICLR) (2022)
67. Zhang, C., Benz, P., Karjauv, A., Sun, G., Kweon, I.S.: Udh: Universal deep hiding for steganography, watermarking, and light field messaging. Advances in Neural Information Processing Systems **33**, 10223–10234 (2020)
68. Zhang, C., Karjauv, A., Benz, P., Kweon, I.S.: Towards robust deep hiding under non-differentiable distortions for practical blind watermarking. In: Proceedings of the 29th ACM international conference on multimedia. pp. 5158–5166 (2021)
69. Zhang, K.A., Xu, L., Cuesta-Infante, A., Veeramachaneni, K.: Robust invisible video watermarking with attention. arXiv preprint arXiv:1909.01285 (2019)
70. Zhang, L., Liu, X., Martin, A.V., Bearfield, C.X., Brun, Y., Guan, H.: Robust image watermarking using stable diffusion. arXiv preprint arXiv:2401.04247 (2024)
71. Zhang, L., Rao, A., Agrawala, M.: Adding conditional control to text-to-image diffusion models. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 3836–3847 (2023)
72. Zhang, R., Isola, P., Efros, A.A., Shechtman, E., Wang, O.: The unreasonable effectiveness of deep features as a perceptual metric. In: Proceedings of the IEEE conference on computer vision and pattern recognition. pp. 586–595 (2018)
73. Zhang, S., Chen, H.S., Pfeiffer, R.M.: A combined p-value test for multiple hypothesis testing. Journal of Statistical Planning and Inference **143**(4), 764–770 (2013)
74. Zhang, X., Karaman, S., Chang, S.F.: Detecting and simulating artifacts in gan fake images. In: 2019 IEEE international workshop on information forensics and security (WIFS). pp. 1–6. IEEE (2019)
75. Zhao, Y., Pang, T., Du, C., Yang, X., Cheung, N.M., Lin, M.: A recipe for watermarking diffusion models. arXiv preprint arXiv:2303.10137 (2023)
76. Zhu, J., Kaplan, R., Johnson, J., Fei-Fei, L.: Hidden: Hiding data with deep networks. In: Proceedings of the European conference on computer vision (ECCV). pp. 657–672 (2018)
77. Zohny, H., McMillan, J., King, M.: Ethics of generative ai (2023)

Table 7: Comparison results of LaWa with existing post- and in-generation image watermarking methods in robustness against an extended set of attacks. *: in-generation methods.

Method (bit#)	Bit accuracy ↑																
	None	Rot. 5	Rot. 15	Rot. 25	C. Crop 0.1	C. Crop 0.3	C. Crop 0.5	C. Crop 0.7	C. Crop 0.9	R. Crop 0.1	R. Crop 0.3	R. Crop 0.5	R. Crop 0.7	R. Crop 0.9	Resize 0.6	Resize 0.7	Resize 0.8
DCT-DWT (32) [16]	0.52	0.51	0.51	0.51	0.50	0.51	0.51	0.51	0.51	0.51	0.51	0.51	0.51	0.51	0.50	0.51	0.51
Hidden (30) [31]	0.91	0.89	0.79	0.62	0.91	0.91	0.91	0.91	0.91	0.91	0.91	0.91	0.91	0.91	0.87	0.89	0.89
SSL (32) [20]	**1.00**	**1.00**	**0.99**	**0.99**	0.74	0.92	0.97	0.99	**1.00**	0.72	0.92	0.98	0.99	**1.00**	0.98	0.99	**1.00**
RoSteALS (32) [10]	0.98	0.82	0.47	0.49	0.52	0.50	0.52	0.68	0.96	0.52	0.52	0.51	0.64	0.90	0.98	0.98	0.98
RivaGan (32) [69]	0.99	0.98	0.91	0.72	0.93	0.98	0.99	0.99	0.99	0.93	0.98	**0.99**	**0.99**	0.99	0.97	0.98	0.99
LaWa* (32)	**1.00**	**1.00**	**1.00**	**0.99**	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	**0.98**	**0.99**	**0.99**	**0.99**	0.99	**0.99**	**1.00**	**1.00**
LaWa-post-gen (32)	**1.00**	**1.00**	**1.00**	0.98	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	0.96	0.98	**0.99**	**0.99**	0.99	0.97	**1.00**	**1.00**
SSL (48) [20]	**1.00**	**0.99**	**0.99**	**0.98**	0.72	0.88	0.96	0.97	0.99	0.70	0.89	0.96	**0.99**	**1.00**	0.96	0.99	0.99
FNNS (48) [32]	**1.00**	**0.99**	0.76	0.58	**0.98**	0.99	**0.99**	**0.99**	**0.99**	**0.96**	**0.98**	**0.99**	**0.99**	**0.99**	0.86	0.95	0.99
RoSteALS (48) [10]	**1.00**	0.56	0.53	0.49	0.51	0.51	0.50	0.49	0.83	0.50	0.50	0.50	0.50	0.76	**1.00**	**1.00**	**1.00**
SS* (48) [19]	0.99	0.96	0.81	0.65	0.95	0.97	0.98	**0.99**	**0.99**	0.91	0.95	0.97	0.98	**0.99**	0.80	0.87	0.94
LaWa* (48)	**1.00**	**0.99**	**0.99**	0.89	0.95	0.98	**0.99**	**0.99**	**0.99**	0.85	0.96	0.98	**0.99**	0.99	0.89	0.98	**1.00**
LaWa-post-gen (48)	**1.00**	**0.99**	0.97	0.83	0.93	**1.00**	**0.99**	**0.99**	**0.99**	0.84	0.90	**0.99**	**0.99**	**0.99**	0.95	0.95	0.98

Method (bit#)	Bit accuracy ↑																
	Resize 0.6	Blur 3	Blur 7	Blur 11	Blur 15	Blur 19	Contr. 0.5	Contr. 1.5	Contr. 2.0	Bright. 0.5	Bright. 1.5	Bright. 2.0	JPEG 10	JPEG 30	JPEG 50	JPEG 70	JPEG 90
DCT-DWT (32) [16]	0.51	0.78	0.56	0.53	0.51	0.51	0.50	0.52	0.50	0.50	0.51	0.50	0.50	0.50	0.51	0.52	0.55
Hidden (30) [31]	0.89	0.84	0.76	0.66	0.56	0.51	0.75	0.78	0.75	0.79	0.77	0.74	0.46	0.48	0.50	0.53	0.64
SSL (32) [20]	**1.00**	**1.00**	**1.00**	**1.00**	0.97	0.84	**1.00**	0.99	0.96	**1.00**	0.99	0.95	0.57	0.83	0.95	0.99	**1.00**
RoSteALS (32) [10]	0.98	0.98	0.98	0.98	0.98	**0.98**	0.77	0.77	0.74	0.80	0.75	0.73	**0.84**	0.95	0.97	0.98	0.98
RivaGan (32) [69]	0.99	0.99	0.99	0.98	0.96	0.91	0.74	0.84	0.81	0.77	0.83	0.80	0.63	0.85	0.94	0.98	0.99
LaWa* (32)	**1.00**	**1.00**	**1.00**	0.99	0.98	**0.98**	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	0.72	**0.97**	**1.00**	**1.00**	**1.00**
LaWa-post-gen (32)	**1.00**	**1.00**	**1.00**	**1.00**	**0.99**	**0.98**	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	0.71	**0.97**	**1.00**	**1.00**	**1.00**
SSL (48) [20]	**1.00**	**1.00**	**1.00**	0.99	0.97	0.83	**1.00**	0.98	0.94	**1.00**	0.99	0.94	0.59	0.82	0.94	0.99	**1.00**
FNNS (48) [32]	**1.00**	0.82	0.54	0.51	0.50	0.50	0.53	0.90	0.88	0.54	0.88	0.85	0.59	0.81	0.89	0.94	0.98
RoSteALS (48) [10]	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	0.88	0.89	0.88	0.92	0.88	0.86	**0.97**	**1.00**	**1.00**	**1.00**	**1.00**
SS* (48) [19]	0.98	0.96	0.78	0.63	0.55	0.50	0.97	0.98	0.97	0.97	0.98	0.96	0.63	0.84	0.89	0.92	0.95
LaWa* (48)	0.99	**1.00**	0.99	0.99	0.96	0.93	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	0.82	0.98	**1.00**	**1.00**	**1.00**
LaWa-post-gen (48)	0.98	**1.00**	0.99	0.98	0.95	0.92	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	0.80	0.98	**1.00**	**1.00**	**1.00**

A Additional Experiments

A.1 Robustness Under Different Attacks

In addition to the results in the main body of the paper, Tab. 7 provides a comprehensive list of different attacks and the robustness of LaWa and other post- and in-generation methods on the AI-Generated dataset. LaWa outperforms other methods in the majority of attacks and has comparable robustness for the other attacks. Furthermore, Tab. 8 presents more detailed performance results of LaWa (48-bit version) in different generative tasks under different attack parameters. It is evident that LaWa has great robustness to common image transformation attacks for all generative tasks.

A.2 General Image Watermarking

In the main body of the paper, the results of LaWa for general image watermarking are reported using CLIC dataset. We also use DIV2K [3] dataset, which has larger variety of scenes compared to CLIC dataset to evaluate the performance of LaWa against baselines. All the 900 images in the training and validation set of Div2K are used in the experiments. For all images, while maintaining the aspect ratio, larger dimension of each image is resized to 1024 to avoid memory

Table 8: Performance of LaWa for different generative tasks and different image modifications attacks.

Generative Task	Bit accuracy ↑										
	None	C. Crop 0.4	C. Crop 0.2	R. Crop 0.4	R. Crop 0.1	Resize 0.7	Blur 11	Contr. 1.5	Bright. 2.0	JPEG 60	Comb.
Inpainting	1.00	0.99	0.98	0.91	0.86	0.97	0.98	1.00	1.00	0.98	0.96
Super-Resolution	1.00	0.98	0.97	0.91	0.90	0.97	0.95	1.00	0.99	0.98	0.92
Image Editing	1.00	0.99	0.98	0.97	0.87	0.97	0.98	1.00	0.98	1.00	0.94
Text-to-Image	1.00	0.99	0.98	0.90	0.85	0.98	0.99	0.99	1.00	1.00	0.94

Table 9: Comparison results of LaWa with existing post-generation image watermarking methods in terms of quality, embedding time, and robustness against various attacks on DIV2K dataset.

Method (bit#)	Image quality		Emb. time (ms)	Bit accuracy ↑										
	PSNR/SSIM ↑	LPIPS/SIFID ↓		None	C. Crop 0.1	R. Crop 0.1	Resize 0.7	Rot. 15	Blur	Contr. 2.0	Bright. 2.0	JPEG 70	Comb.	Ave.
DIV2K														
DCT-DWT (32) [16]	36.73/0.97	**0.01**/**0.01**	224	0.89	0.50	0.50	0.50	0.51	0.51	0.50	0.50	0.51	0.50	0.54
Hidden (30) [31]	32.34/0.95	0.04/0.06	25	0.89	0.89	0.89	0.86	0.76	0.66	0.75	0.73	0.56	0.62	0.76
SSL (32) [20]	35.04/0.94	0.10/0.11	1181	**1.00**	0.83	0.78	**1.00**	**1.00**	**1.00**	0.97	0.95	0.99	0.90	0.94
RoSteALS (32) [10]	25.91/0.88	0.09/0.07	236	**1.00**	0.50	0.50	**1.00**	0.50	**1.00**	0.90	0.87	**1.00**	0.50	0.78
RivaGan (32) [69]	**42.16**/**0.98**	0.03/0.04	81	**1.00**	0.97	0.95	0.99	0.94	0.99	0.86	0.84	0.98	0.96	0.95
LaWa (32)	35.19/0.92	0.05/0.04	34	**1.00**	**1.00**	**0.96**	0.97	0.99	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	**0.99**
SSL (48) [20]	35.04/0.94	0.10/0.11	1181	**1.00**	0.81	0.76	**0.99**	**0.99**	**1.00**	0.96	0.95	0.98	0.88	0.93
FNNS (48) [32]	**37.41**/**0.97**	**0.03**/0.08	1003	**1.00**	**1.00**	**0.97**	0.93	0.75	0.51	0.85	0.81	0.90	0.90	0.86
RoSteALS (48) [10]	25.62/0.88	0.09/**0.07**	236	0.99	0.50	0.50	**0.99**	0.47	0.99	0.82	0.80	**0.99**	0.50	0.76
LaWa (48)	34.57/0.91	0.07/0.08	34	**1.00**	**1.00**	0.95	0.97	**0.99**	0.99	**1.00**	**1.00**	**0.99**	**0.91**	**0.98**

limitation issue for some of the baselines. The quality and the robustness for LaWa as well as the baselines are shown in Tab. 9. LaWa outperforms baseline methods in the majority of the attacks. For image quality, it has comparable perceptual quality to the baseline methods. In particular, for 48-bit scenario, FNNS has higher PSNR, but it suffers from significantly high computational complexity as well as low robustness against many attacks such as blur, contrast, and brightness.

A.3 Ablation Studies

In this section, more ablation analysis over different components of the proposed method is provided.

Different Loss Terms. We study the effect of different loss terms on the perceptual quality of generated images in the main body of the paper and show that using $\mathcal{L}_{MSE}$, $\mathcal{L}_{LPIPS}$, and $\mathcal{L}_{Critic}$ together achieves the best performance. In Fig. 8, the effect of each of these loss terms on the watermarked images is qualitatively illustrated. When $\mathcal{L}_{Critic}$ is not used, visual artifacts are present in the image. Adding $\mathcal{L}_{Critic}$ improves the quality of the image. Furthermore, using $\mathcal{L}_{LPIPS} + \mathcal{L}_{Critic}$ is more effective than $\mathcal{L}_{MSE} + \mathcal{L}_{Critic}$, while best perceptual quality is achieved when three losses are used together.

Watermark Block Size. Table 10 shows the effect of changing the size B of noise block $b_i \in \mathbb{R}^{B \times B \times C_i}$ using 10 messages (48 bits) and 1k images

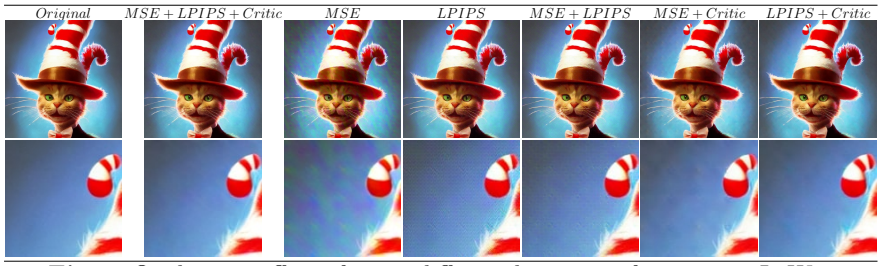


Fig. 8: Qualitative effect of using different loss terms for training LaWa.

Table 10: Effect of the watermark block size.

Block size (B)	PSNR/SSIM ↑	LPIPS/SIFID ↓	None	C. Cro 0.1	Contr. 2.0	Rot. 15	JPEG 70
2	30.88/0.66	0.07/0.21	0.99	0.85	0.98	0.82	0.97
4	32.36/0.81	0.06/0.07	0.99	0.90	0.99	0.82	0.95
8	33.52/0.86	0.04/0.02	**1.00**	**0.95**	**1.00**	**0.96**	**1.00**
16	34.43/0.88	0.03/0.02	0.99	0.38	0.98	0.48	0.96
32	**34.51/0.91**	**0.02/0.02**	0.90	0.56	0.86	0.45	0.83

per message. As we increase B, the perceptual quality of the generated images improves. However, when B is set to 16 or 32, LaWa is not robust to rotation and crop attacks anymore. Thus, we use $B = 8$ in all our experiments.

Number of Embedding Modules. Table 11 illustrates the effect of increasing the number of watermarking modules from 1 to 4, showing an improvement in both robustness and perceptual quality. Specifically, adding more modules significantly improves the robustness to crop and rotation attacks.

A.4 Other Auto-Encoder Networks

As discussed in the main text, LDM's image decoder $\mathcal{D}$ upsamples the generated latent to image by a factor f, where $f \in \{4, 8, 16, 32\}$. Furthermore, different decoder networks such as KL-VAE [47] and VQ-VAE [18] can be used for LDMs. To show the generality of LaWa to different networks and downsampling factors, we train LaWa using different auto-encoders and factors. The corresponding results are shown in Tab. 12. For KL-f8 model, we use 10 messages and generate 1K images per message using LDM model. For other models, we use the frozen encoder of each model to map 1K images to the latent space. We then reconstruct one image using the original decoder and one image using the modified decoder. Each image is watermarked using a random 48-bit message.

Table 11: Effect of number of embedding modules.

W_{Emb_0}	W_{Emb_1}	W_{Emb_2}	W_{Emb_3}	PSNR/SSIM ↑	LPIPS/SIFID ↓	None	C. Cro 0.1	Contr. 2.0	Rot. 15	JPEG 70
✓				30.75/0.79	0.07/0.08	**1.00**	0.90	0.99	0.90	0.99
✓	✓			32.83/0.80	0.04/0.03	**1.00**	0.94	1.00	0.94	0.99
✓	✓	✓		32.6/0.82	0.04/0.02	0.99	0.94	**1.00**	**0.96**	**1.00**
✓	✓	✓	✓	**33.52/0.86**	**0.04/0.02**	**1.00**	**0.95**	**1.00**	**0.96**	**1.00**

As shown in Tab. 12, LaWa achieves high perceptual quality and high robustness against attacks in all scenarios, which shows the applicability of LaWa to different auto-encoders and downsampling factors. It should be noted that as the downsampling factor increases, the perceptual quality of the generated images as well as the bit extraction accuracy experience minor degradation. This is due to more compression of the latent providing less information for generating and watermarking the images.

A.5 Theoretical vs. Empirical False Positive Rate

Assuming that extracted bits from vanilla images are independent and identically distributed (i.i.d) Bernoulli random variables with parameter 0.5, a theoretical upper bound for the FPR of detection and attribution can be defined.

Detection. If we use LaWa only for the detection problem, all images can be generated with the same message m. To calculate the FPR in this problem, we test the hypothesis $H_1 : M(\hat{m}, m) \geq n$ against the null hypothesis H_0: "any random message, $\tilde{m}$, match with m in more than n bits". Under the null hypothesis, the probability of success follows a binomial distribution where total number of trials is k, number of successes is n, and the probability of a match for each bit is 0.5. The calculated probability from the binomial distribution corresponds to the upper bound of the FPR, which happens if W_{Ext} extracts random bits. Therefore, for each n, we can obtain the FPR of detection, $FPR_{det}(n)$, using the CDF of the binomial distribution:

$$FPR_{det}(n) = Pr\big(M(\tilde{m}, m) > n \mid H_0\big) = \sum_{i=n+1}^{k} \binom{k}{i} 0.5^k. \tag{9}$$

By setting the FPR_{det} at a desired level, we can use Eq. (9) to obtain the minimum required n to be used for watermark matching.

Attribution. In the attribution problem, we attribute the image to a specific user by matching $\hat{m}$ with one of the messages in $\{m_1, ..., m_N\}$, where N is the total number of users. If $M(\hat{m}, m_i) \geq n$, then the ith user is flagged as the creator of the image. The watermark matching is repeated for all N users, which corresponds to having N hypothesises. The attribution FPR, $FPR_{att}(N, n)$, or the probability of a random message, $\tilde{m}$, to match with at least one message in $\{m_1, ..., m_N\}$ is the global p-value [73]:

$$FPR_{att}(N, n) = 1 - (1 - FPR_{det}(n))^N. \tag{10}$$

The detection and attribution FPRs are calculated with the assumption that the extracted bits from vanilla unwatermarked images are i.i.d Bernoulli random variables. However, the i.i.d assumption may not hold for our trained watermark extractor, meaning that extracted watermark bits may not be independent. Therefore, we empirically calculate the detection false positive rate. For 1 million vanilla images selected from the training set of ImageNet dataset [17], we resize the images to 512×512 and extract a watermark from each vanilla image $m_{vanilla}$. We then randomly generate 100 different ground truth watermark

Table 12: Performance of LaWa with different auto-encoder networks and downsampling factors.

Model	PSNR/SSIM ↑	LPIPS/SIFID ↓	None	C. Cro 0.1	Contr. 2.0	Rot. 15	JPEG 70
KL-f4	35.21/0.93	0.03/0.03	1.00	0.96	1.00	0.94	0.97
KL-f8	33.52/0.86	0.04/0.02	1.00	0.95	1.00	0.96	1.00
KL-f16	32.87/0.85	0.06/0.03	1.00	0.93	0.98	0.91	0.95
VQ-f4	36.06/0.94	0.02/0.01	1.00	0.91	1.00	0.90	0.98
VQ-f8	34.21/0.92	0.04/0.02	1.00	0.90	0.99	0.89	0.98
VQ-f16	33.15/0.88	0.07/0.02	0.99	0.88	0.97	0.85	0.95

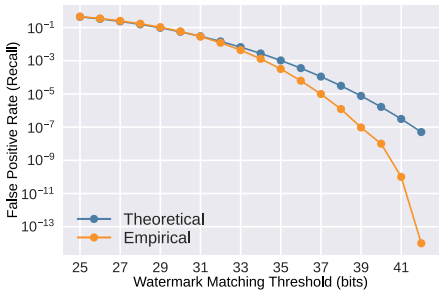


Fig. 9: Comparison of the theoretical and empirical false positive rate for the detection task using 48-bit watermarks and different watermark matching thresholds (n).

messages m_{gt}. The watermark matching threshold n is selected from $\{25, ..., 42\}$. For each pair of watermark message and watermark matching threshold, we perform watermark matching between the 1 million extracted messages and the ground truth watermark message and calculate the FPR. False positive corresponds to vanilla images that their message matches the ground truth message ($M(m_{vanilla}, m_{gt}) \geq n$). Figure 9 compares the FPR for both theoretical and empirical cases. The empirical rate is the average over the false positive rate of all 100 messages. For thresholds above 33, the empirical false positive rate is below the theoretical one. This is good result and shows that LaWa achieves lower rate of falsely flagging vanilla images as watermarked compared to the theoretical rate. For example for $n = 41$, LaWa flags only 1 of 10 million vanilla image as watermarked ($FPR_{empirical} = 10^{-10}$ vs $FPR_{theoretical} = 10^{-7}$).

A.6 Additional Qualitative Analysis

In this section, we provide more qualitative results of LaWa's performance.

Qualitative Comparison to Other Methods. Figures 10 and 11 compare the performance of LaWa to other image watermarking methods for 48-bit and 32-bit watermark messages, respectively. Images are generated with resolution 512×512. For the examples provided in both fig-ures, the original, residual (multiplied by 10), and cropped (for better visibility) images are shown.

As shown in the residual images in Fig. 10 (the 48-bit version), although FNNS seems to provide the lowest amount of noise in the solid backgrounds, it

still suffers from the large artifacts on the edges of the objects (e.g., the girl's face). Such large artifact blocks are also seen for RoSteALS. SSL, on the other hand, gives the worst quality compared to the others, while it tends to provide uniformly distributed small noises all around the images. From the watermarked images, the best quality is presented by LaWa and Stable Signature, but still LaWa adds less noise, especially in the solid backgrounds.

From the cropped images in Fig. 11 (the 32-bit version), SSL's low quality is still clear. From the residual images, RivaGAN and DCT-DWT seem to have the lowest noise added to the generated images. However, some red stripped noise lines on the images watermarked by RivaGAN are evident (e.g., on the girl's face as well as the background). Despite the best quality given by DCT-DWT, it has a very low robustness against attacks as described in the main text. Hidden's quality is also shown to be bad from both the residual and cropped images as it adds too much noise during the watermarking. In general, LaWa and RoSteALS show the best perceptual quality compared to the others.

Figure 12 provides a qualitative comparison between watermarked images from the CLIC [56] dataset. Furthermore, more qualitative results from LaWa are illustrated in Fig. 13 including the original and watermarked images as well as their residuals. Images are generated using prompts from MS-COCO and MagicPrompt-SD datasets with a resolution of 512×512.

B Implementation Details

B.1 Training Details

At each training step, an image transformation attack T is randomly selected from $\mathcal{T}$, which includes the following:

1. Center area crop (C. Crop) with areas between 8% to 95%
2. Random area crop (R. Crop) with areas between 8% to 95%
3. Rotation (Rot.) with angles between 2 and 46 degrees
4. Area resizing with areas between 0.5 and 1.5
5. Brightness (Bright.) with factors between 0.0 and 3.0
6. Gaussian noise with standard deviations between 0 and 0.05
7. Blur with kernels between 3 and 19
8. Contrast (Contr.) with factors between 0.0 and 3.0
9. JPEG compression with compression factors between 40 and 100
10. Identity

The parameter for each attack is randomly sampled from a uniform distribution between the lowest and highest parameter. At the training time, we start applying attacks after the bit accuracy reaches a certain threshold, set to 75%. Furthermore, training for 40 epochs using 8 GPUs with 32GB of memory takes 2 days to finish.

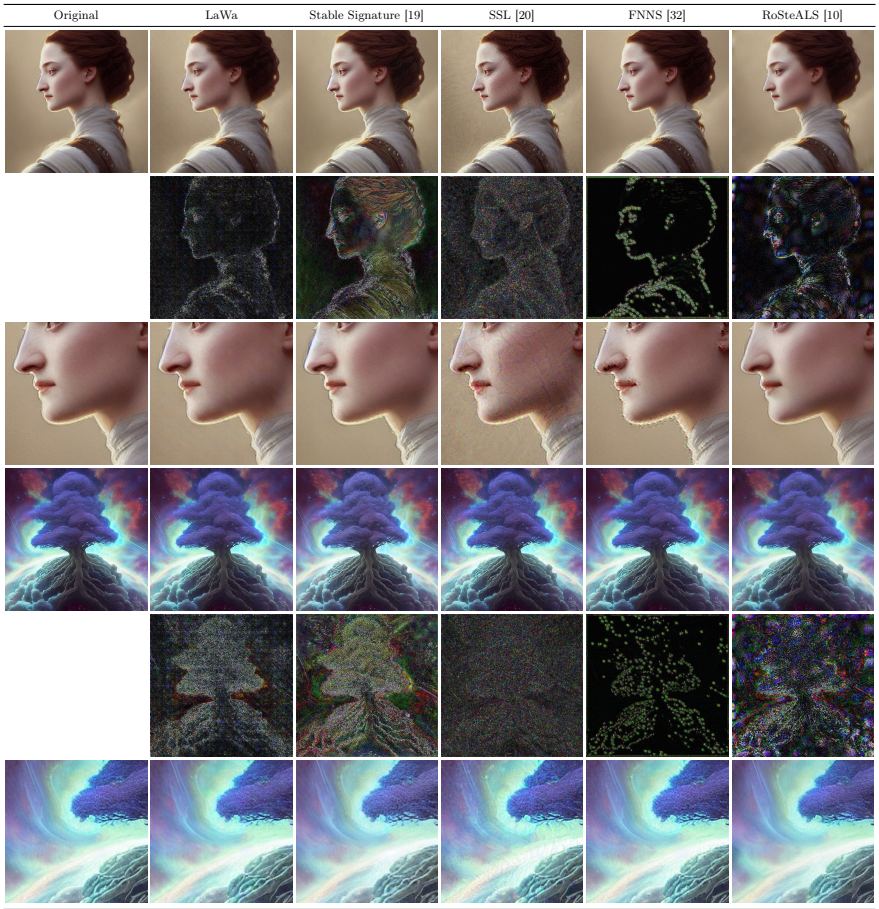


Fig. 10: Qualitative comparison of watermarked images using LaWa and other methods for 48-bit watermarks.

B.2 More Details on Experiments Settings

For all the experiments related to the text-to-image task, we use Stable Diffusion version 1.4 [1]. However, for the other image generation tasks (i.e., editing, inpainting, and super-resolution), Stable Diffusion version 2.1 [2] is employed. For all the tasks, their corresponding default parameters of the Stable Diffusion models are used. In all experiments at the test/inference time, we use images with 512×512 resolution. For all tasks except text-to-image, we create a set of 1K random 48-bit messages and create one image per message. For text-to-image task, we use 10 different messages and create 1K images per message.

B.3 Image Generation Tasks

More details about each task is provided below. **Image editing.** In this task, an existing image is modified by the diffusion process using a new text description.

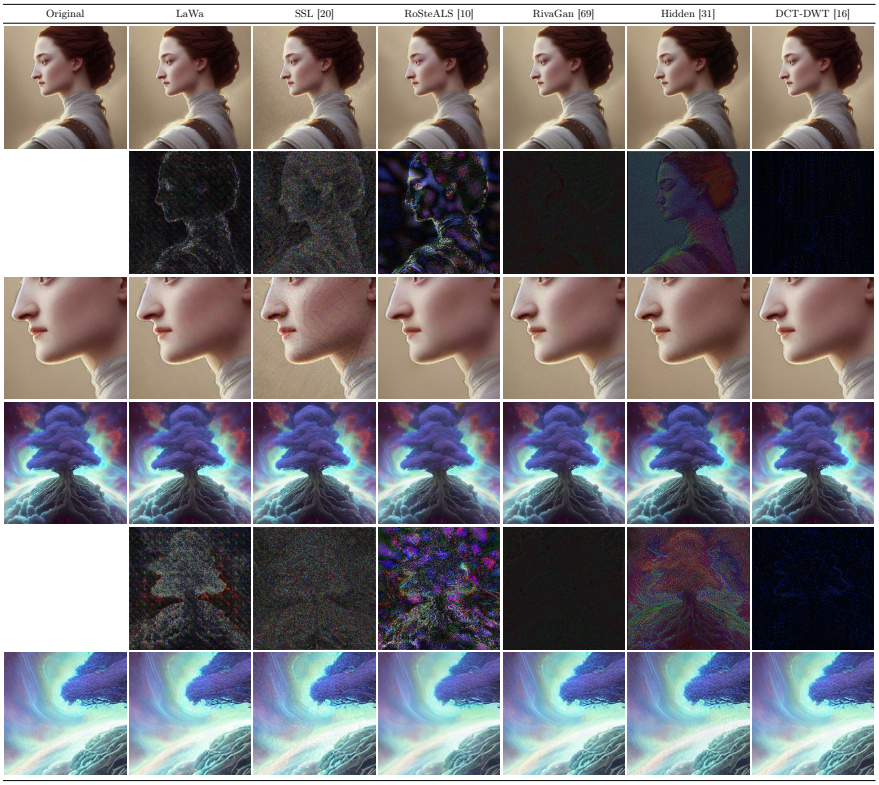


Fig. 11: Qualitative comparison of watermarked images using LaWa and other methods for 32-bit watermarks.

We follow [15] and use the validation set of MS-COCO [36] to generate 1k pairs of image and novel descriptions that the new generated image should follow.

Inpainting. In this task, some parts of an image is covered by a mask and the diffusion process inpaints the hidden parts behind the mask. For this task, we follow the protocol of LaMa [54] for 1k images selected from MS-COCO. We use "thick" setting of the protocol and create masks that randomly cover 1% to 50% of the image.

Super-resolution. In this task, diffusion process is used to increase the resolution of an existing image. We resize 1k images from MS-COCO to 128×128 resolution using bicubic interpolation. Diffusion process scales the resolution of the images back to 512×512.

Text-to-image. In this task, a text prompt guides the diffusion process. We use 500 prompts randomly selected from MS-COCO and 500 prompts randomly selected from MagicPrompt-SD [49]. Using 10 random messages, we generate a total of 10k images for this task.

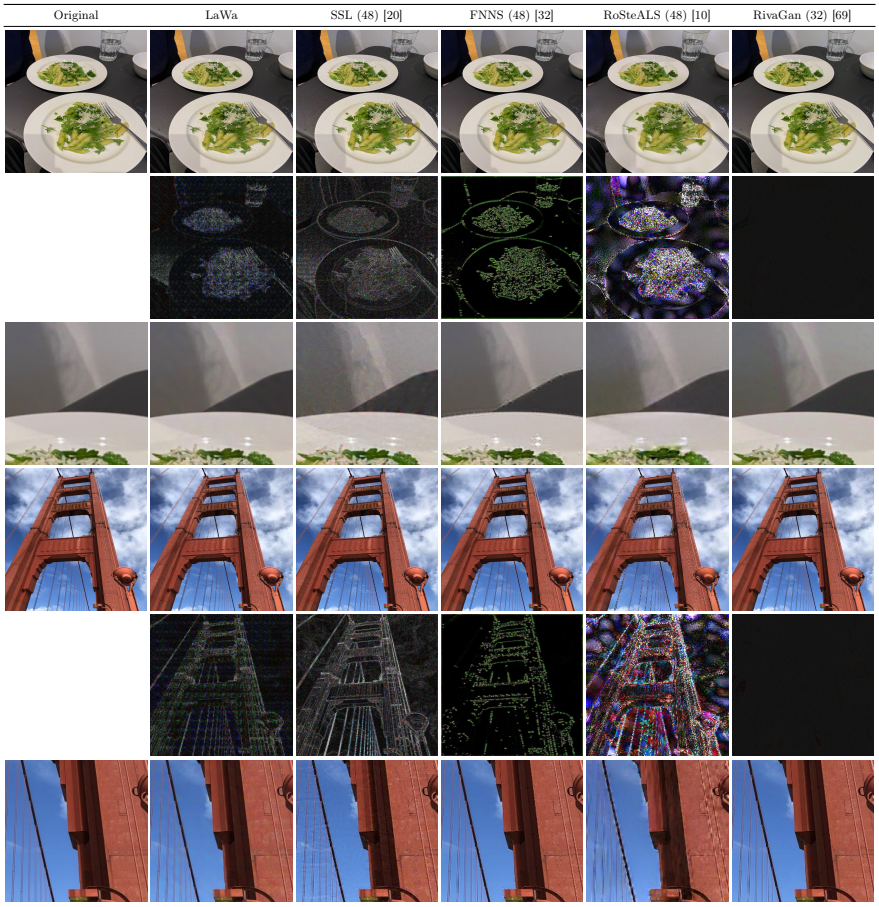


Fig. 12: Qualitative comparison of watermarked images using LaWa and other methods on natural images from CLIC [56] dataset.

B.4 Image Watermarking Methods

For DCT-DWt, we used the implementation in [52]. For SSL watermark [20], we set the number of optimization steps to 40 and the PSNR to 33. We used the provided pre-trained weights of the method for feature extractor network. For FNNS [32], we set the number of iterations to 10 and use the decoder of Stable Signature [19] as the network used for the optimization process of FNNS. For RoSteaALS [10], the public pre-trained weights are for 100-bit watermarks. Therefore, we train two 32-bit and 48-bit versions of the method using the same dataset used for LaWa. Crop attack is not included in the default training process of RoSteaALS and we add this attack during the training. We use the default auto-encoder network used by RoSteaALS. For RivaGan [69] and HiDDen [76], we use the pre-trained weights provided in [52] and [31], respectively. For Stable Signature [19], we follow the provided public code and fine-tune 10 KL-f8 auto-encoders using 10 different messages and create 1k images with each message.

B.5 Learning-Based Attack

In this attack, the watermarked image is passed through a pre-trained auto-encoder network in order to remove the watermark information from the reconstructed image by the auto-encoder. We select image compression auto-encoders Bmshj2018 [6] and Cheng2020 [12] and use the weights available in CompressAI library zoo [7]. We use $q = \{4, 8\}$ and $q = \{3, 6\}$ for Bmshj2018 and Cheng2020, respectively. We also use image generation auto-encoders KL-VAE [47] with $f = \{4, 8, 16, 32\}$ and VQ-VQE [18] with $f = \{4, 8, 16\}$. For these models, the publicly available pre-trained weights of [1] are used.

C Discussion of Limitations

LaWa addresses the problem of black-box image watermarking, where we assume users do not have access to the generative models. Thus, the main application scenario of LaWa is focused on image generation services. However, model owners may decide to make their models public, where users have white-box access to model weights. Although our intermediate watermarking modules are integrated as extra layers to the model, a hostile user may be able to remove these modules and achieve image generation without watermark. Furthermore, imperceptible image watermarking has generally limited robustness to newer learning-based attacks. We show that LaWa is robust to such attacks for some models. However, with the rapid development of image modification models, a hostile user can use such models to evade the watermark. In addition, learning-based image watermarking methods such as LaWa generally need to apply existing differentiable image distortions at training time. As a result, they may have a poor performance on unknown distortions and image modifications.

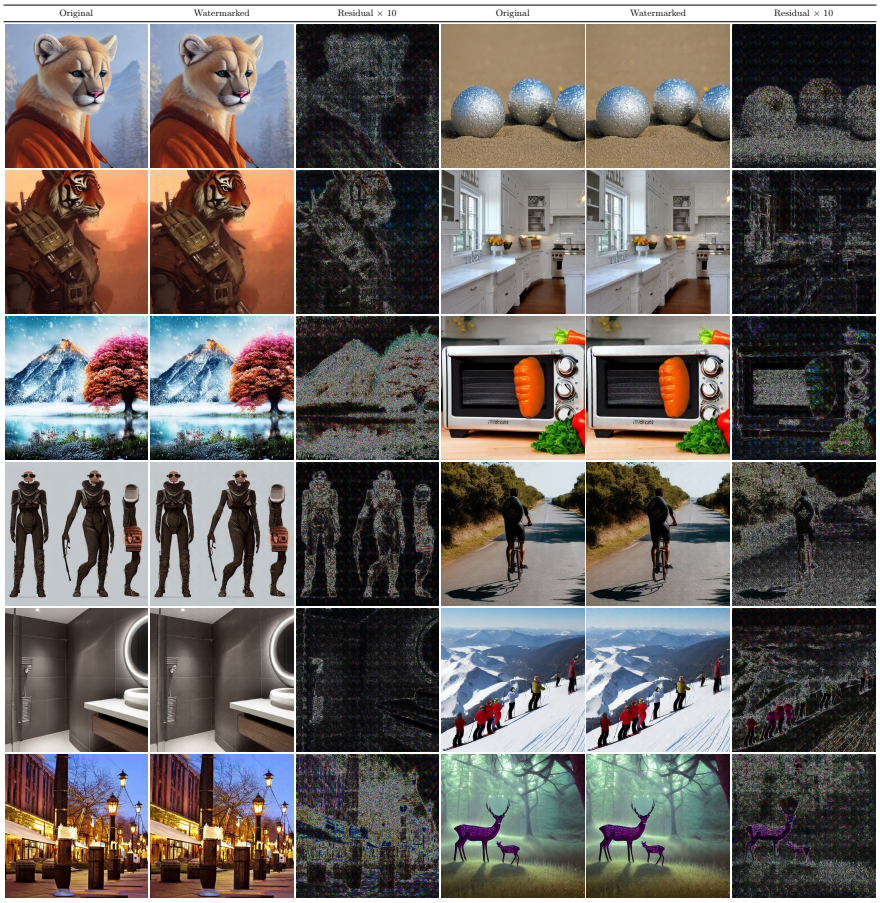


Fig. 13: More qualitative comparison between the original images and the LaWa watermarked images along with their residuals (multiplied by 10).